Supplement to

Generators and Relations for Discrete Groups

Hilbert Reconstruction and Lean Formalization

Jurek Różański

Military University of Technology, Warsaw, Poland

CGJteamLab

August 28, 2026

Document Status

This volume is a research supplement accompanying the CGJteamLab formalization project. Its purpose is to investigate the generators-and-relations viewpoint of Coxeter and Moser inside a synthetic Hilbert geometry formalized in Lean.

The source book is H. S. M. Coxeter and W. O. J. Moser, *Generators and Relations for Discrete Groups*, second edition.

It is important to state precisely how that book is being used here. Coxeter–Moser is not primarily a synthetic geometry text. Its organizing language is the algebra of finitely generated discrete groups through generators, relations, abstract definitions, and presentations. Reflection groups form one important family among several treated in the book. The preface places them alongside groups of low order, permutation groups, groups of congruent transformations, symmetric and alternating groups, and linear fractional groups; Chapter 9 then develops the Euclidean reflection-group family in arbitrary dimension.

The present supplement therefore does not reproduce the direction of the source book. Instead, it deliberately reverses part of it. Coxeter–Moser start from algebraic presentations and study, among other things, geometric realizations by reflections. The CGJteamLab program starts from synthetic Hilbert geometry, reconstructs reflections as transformations, and asks which generator relations are forced by the geometry.

The present supplement does not reproduce the source book. Instead, it develops a parallel formal program:

$$\text{Hilbert geometry} \longrightarrow \text{geometric transformations} \longrightarrow \text{generators} \longrightarrow \text{words} \longrightarrow \text{relations}.$$

The Lean source is authoritative for theorem statements, assumptions, dependencies, and proof status. This document records the mathematical architecture, formal dependency structure, and relation to the Coxeter–Moser viewpoint.

At the present checkpoint the reflection-generator and word layers are implemented, and two finite pairwise cases have been reached synthetically. For perpendicular axes,

$$(r_1 r_2)^2 = 1,$$

and for two median reflection axes of an equilateral triangle,

$$r_a r_b r_a = r_b r_a r_b, \qquad (r_a r_b)^3 = 1.$$

The $p = 3$ theorem is currently established pointwise; its final export to the packaged word-layer predicate `ReflectionPairRelation` is the next local checkpoint.

Contents

Research Program

The initial research program is deliberately narrow.

1. Reconstruct reflection in a line from the established Hilbert incidence, order, congruence, midpoint, and right-angle theory.
2. Promote the geometric reflection relation to an actual permutation of the point set.
3. Prove the first Coxeter relation
$$r_l^2 = 1.$$
4. Construct words in several reflection generators.
5. Study the pairwise relations
$$(r_l r_m)^p = 1$$
for intersecting axes.
6. Treat the parallel-axis case, corresponding to infinite dihedral behavior and translation.
7. Use selected propositions of Euclid's *Elements* as benchmark configurations for the generators-and-relations language.

Items 1–4 are complete. Item 5 has now been entered through two synthetic finite instances: perpendicular axes give $p = 2$, while the median axes of an equilateral triangle give the first noncommutative case $p = 3$. The general finite-period problem remains open.

The principal Coxeter–Moser source locations for the opening phase are Sections 1.1, 4.1, 4.3, and 9.1–9.3.

Chapter 1

Line Reflection as a Hilbert Generator

1.1 Purpose and Status

This document records the first completed component of the experimental Coxeter layer of CGJteamLab. The source module is

```
CGJteamLab/Coxeter/Reflection.lean
```

The module is deliberately separated from the flat sequence of Euclid Proposition files. It is not another proposition of the *Elements*; it is a transformation layer derived over the same Hilbert foundation.

The immediate objective is to obtain a genuine reflection generator

$$r_l : \text{Point} \longrightarrow \text{Point}$$

without introducing reflection as a new primitive geometric operation. The construction starts with a geometric relation, proves existence and uniqueness, and only then packages the relation as a point transformation.

The current file compiles cleanly. No new geometric axiom and no `sorry` is introduced locally.

1.2 Coxeter Motivation

Coxeter and Moser begin with generators and defining relations and later specialize to groups generated by reflections. In the reflection-group language, each basic reflection is involutory:

$$r_i^2 = 1,$$

while the geometry of pairs of reflecting hyperplanes is encoded by the periods of products

$$(r_i r_j)^{p_{ij}} = 1.$$

Their graphical notation records these pairwise periods by a Coxeter diagram.

The present module addresses only the first relation. It constructs a single Hilbert line reflection and proves, in literal operator form,

$$r_l(r_l(P)) = P.$$

Products of distinct reflections and relations of the form $(r_l r_m)^p = 1$ belong to the next layer and are intentionally absent from this file.

The relevant source locations in Coxeter–Moser, *Generators and Relations for Discrete Groups*, second edition, are Sections 1.1, 4.3, and 9.1–9.3.

1.3 Geometric Design

1.3.1 Why the construction begins as a relation

The Hilbert development does not take a global transformation space as a primitive object. The natural first notion is therefore a relation between a point P and its reflected point P'.

For a point off the axis l, reflection is characterized by a foot H satisfying

$$H \in l, \qquad P - H - P', \qquad PH \cong HP',$$

together with the condition that PH is perpendicular to the axis.

For a point on the axis, the reflected point is the point itself.

Thus the geometric core is

$$\boxed{P' = P \text{ on } l \quad \text{or} \quad H \in l,\ H = \mathrm{mid}(P, P'),\ PH \perp l.}$$

1.3.2 ReflectionAxis

The current Hilbert API often represents a line geometrically by two distinct points known to lie on it. The reflection axis therefore stores both the carrier and witnesses sufficient for perpendicular constructions.

```
structure ReflectionAxis
    [HilbertIncidence Geo] where
  carrier : Geo.Line
  A : Geo.Point
  B : Geo.Point
  hAB : Not (A = B)
  hA : HilbertIncidence.OnLine A carrier
  hB : HilbertIncidence.OnLine B carrier
```

This is not additional geometry. It packages the nondegenerate line data already required by the existing Hilbert construction interface.

1.3.3 PerpendicularToAxis

Perpendicularity to the axis is represented using the established point-based right-angle language.

```
def PerpendicularToAxis
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (H P : Geo.Point) : Prop :=
  And
    (HilbertIncidence.OnLine H axis.carrier)
    (And
      (Not (HilbertIncidence.OnLine P axis.carrier))
      (Exists fun R : Geo.Point =>
        And
          (HilbertIncidence.OnLine R axis.carrier)
          (And
            (Not (R = H))
            (HilbertRightAngle Geo R H P))))
```

The witness R supplies a nondegenerate direction of the carrier. No primitive relation "two lines are perpendicular" is added.

1.3.4 IsLineReflection

The reflection relation has a fixed-point branch and an off-axis branch.

```
def IsLineReflection
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P P' : Geo.Point) : Prop :=
  Or
    (And
      (HilbertIncidence.OnLine P axis.carrier)
      (P' = P))
    (And
      (Not (HilbertIncidence.OnLine P axis.carrier))
      (Exists fun H : Geo.Point =>
        And
          (PerpendicularToAxis Geo axis H P)
          (HilbertIsMidpoint Geo H P P')))
```

The midpoint condition packages both strict betweenness $P - H - P'$ and equality of the two half-segments.

1.4 Existence of the Reflected Point

The first main theorem is

```
theorem line_reflection_exists
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P : Geo.Point) :
```

```
    exists P' : Geo.Point,
      IsLineReflection Geo axis P P'
```

The proof splits on whether P lies on the axis.

If $P \in l$, choose $P' = P$.

If $P \notin l$, Euclid I.12 in its Hilbert reconstruction supplies the perpendicular foot H. The established theorem

```
hilbert_extend_segment_beyond
```

then extends PH beyond H to a point P' satisfying

$$P - H - P', \qquad PH \cong HP'.$$

This is exactly the midpoint condition needed by `HilbertIsMidpoint`.

The conceptual dependency is therefore

$$\boxed{\text{perpendicular construction} + \text{equal segment extension} \Longrightarrow \text{existence of reflection.}}$$

1.5 Relational Involution

Before constructing a function, the development proves that the reflection relation is symmetric:

```
theorem line_reflection_symmetric
    ...
    (hRef : IsLineReflection Geo axis P P') :
    IsLineReflection Geo axis P' P
```

For off-axis points, the same midpoint H is used in the reverse direction. The substantive geometric obligation is to transport the right-angle condition from the ray HP to the opposite ray HP'. This uses the already developed theory of right-angle transport and opposite extensions.

This theorem is the relational precursor of the Coxeter involution relation:

$$P \; R_l \; P' \Longrightarrow P' \; R_l \; P.$$

1.6 Fixed Points

The module records the exact fixed-point set of the reflection:

```
theorem line_reflection_fixed_iff_on_axis
    ...
    (P : Geo.Point) :
    IsLineReflection Geo axis P P <->
      HilbertIncidence.OnLine P axis.carrier
```

Thus

$$r_l(P) = P \quad \Longleftrightarrow \quad P \in l$$

once the functional reflection has been constructed.

The companion theorem `line_reflection_moves_off_axis` states that an off-axis point cannot be fixed.

1.7 Right-Angle Carrier Transport

The uniqueness proof exposed a reusable gap in the permanent Hilbert API: a right angle should not depend on which non-vertex point is chosen on the same carrier of one arm.

Three local theorems were therefore established:

```
coxeter_right_angle_sameRay_first
coxeter_right_angle_opposite_first
coxeter_right_angle_collinear_first
```

The first transports a right angle along the same ray. The second reverses the first arm across the vertex. The third combines the two cases using betweenness trichotomy and gives the carrier-level statement required for perpendicular uniqueness.

This is a useful example of the formal difference between an intuitive statement about perpendicular lines and the point/ray representation used in the Hilbert layer.

1.8 Uniqueness of the Perpendicular Foot

The key geometric lemma is

```
theorem perpendicular_foot_unique
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P H K : Geo.Point)
    (hPerpH : PerpendicularToAxis Geo axis H P)
    (hPerpK : PerpendicularToAxis Geo axis K P) :
    H = K
```

The proof is neutral.

Assume $H \neq K$. Because both H and K lie on the reflection axis, carrier transport converts the two perpendicularity witnesses into right angles

$$\angle KHP \quad \text{and} \quad \angle HKP.$$

Hence the nondegenerate triangle HKP would have right angles at both H and K.

Extend KH beyond H. The resulting exterior angle is again right. All right angles are congruent, so this exterior angle is congruent to the remote interior right angle at K. Hilbert's exterior-angle theorem forbids this. Therefore $H = K$.

The argument uses no Euclidean parallel axiom. Its mathematical core is

$$\boxed{\text{two distinct perpendicular feet} \Longrightarrow \text{triangle with two right angles} \Longrightarrow \bot.}$$

1.9 Uniqueness of Reflection

Once the perpendicular foot is unique, uniqueness of the reflected point follows from the existing segment-construction uniqueness theorem.

```
theorem line_reflection_unique
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P P1 P2 : Geo.Point)
    (hRef1 : IsLineReflection Geo axis P P1)
    (hRef2 : IsLineReflection Geo axis P P2) :
    P1 = P2
```

In the off-axis case, both reflections have the same perpendicular foot H. Both points P_1, P_2 lie on the prescribed ray beyond H, and their distances are determined by the same midpoint data. Segment additivity and `hilbert_segment_construction_unique` identify the two endpoints.

Thus

$$\forall P\ \exists!\, P'\ \ \text{IsLineReflection}(l, P, P').$$

This is the logical threshold at which reflection can be promoted from a relation to a point transformation.

1.10 The Reflection Operator

The actual point transformation is selected noncomputably from the existence theorem:

```
noncomputable def lineReflect
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P : Geo.Point) :
    Geo.Point :=
  Classical.choose
    (line_reflection_exists Geo axis P)
```

Its specification theorem is

```
theorem lineReflect_spec
    ...
    (P : Geo.Point) :
    IsLineReflection Geo axis P
      (lineReflect Geo axis P)
```

The use of `Classical.choose` is a packaging step only. The geometric existence and uniqueness were proved before the function was introduced.

1.11 The First Coxeter Relation

The main result of the module is

```
theorem lineReflect_involutive
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P : Geo.Point) :
    lineReflect Geo axis
      (lineReflect Geo axis P) = P
```

The proof has a short conceptual form:

$$
\begin{aligned}
& P \; R_l \; r_l(P) \\
\Downarrow \quad & \text{symmetry} \\
& r_l(P) \; R_l \; P \\
& r_l(P) \; R_l \; r_l(r_l(P)) \\
\Downarrow \quad & \text{uniqueness} \\
& P = r_l(r_l(P)).
\end{aligned}
$$

Consequently the first Coxeter relation has now been derived inside the Hilbert model:

$$\boxed{r_l^2 = 1.}$$

This is stronger architecturally than merely assuming an abstract involutory generator: the generator itself has been reconstructed from incidence, order, congruence, midpoint, and right-angle geometry.

1.12 Permutation Packaging

The reflection is finally packaged as a permutation of the point set:

```
noncomputable def lineReflectEquiv
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo) :
    Equiv Geo.Point Geo.Point
```

Both the forward and inverse maps are `lineReflect`; the two inverse laws are exactly `lineReflect_involutive`.

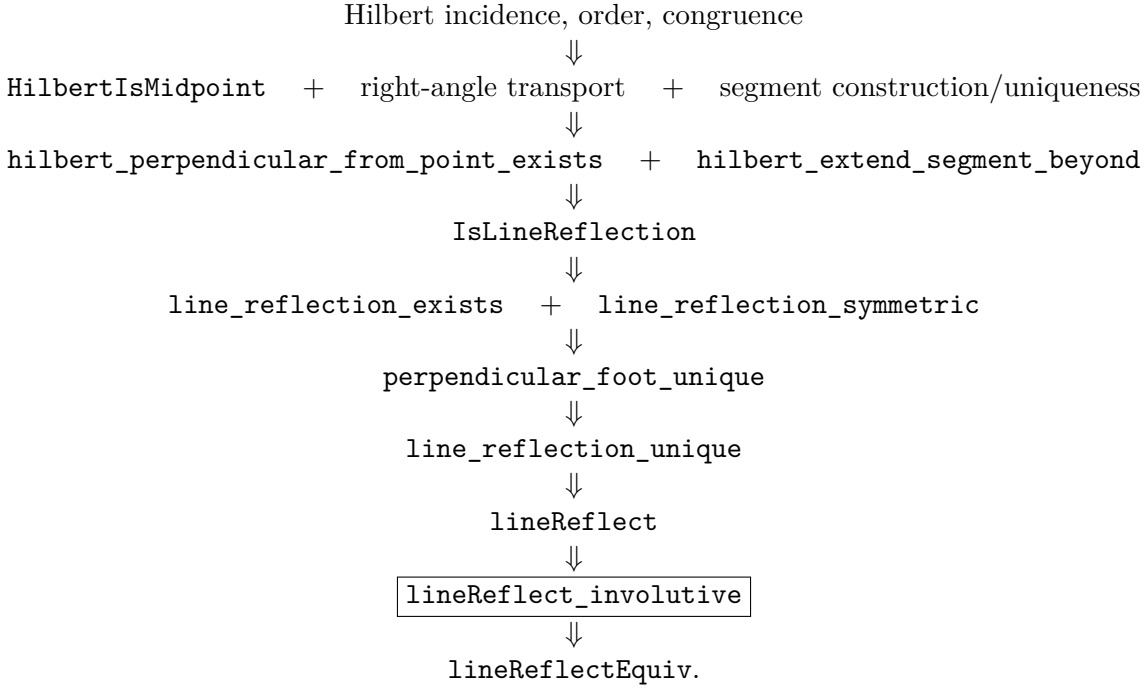
The associated simplification lemmas are

```
lineReflectEquiv_apply
lineReflectEquiv_apply_twice
```

This packaging is the interface needed by the next module, where different reflection generators can be composed into words.

1.13 Dependency Summary

The formal architecture may be summarized as follows:



The only direct Euclid Proposition import required by this module is the Hilbert reconstruction of I.12 for the perpendicular construction. The remaining work belongs to the established Hilbert congruence/order/right angle infrastructure and the local Coxeter helper layer.

1.14 Axiomatic Status

The module introduces no new declared geometric axiom and no `sorry`. The uniqueness of the perpendicular foot is proved by neutral angle theory rather than by uniqueness of parallels.

The transformation `lineReflect` is declared `noncomputable` because it uses `Classical.choose` after existence has been established. This should not be confused with a new geometric axiom.

The exact transitive axiom set should be recorded with `#print axioms` when the Coxeter checkpoint is audited. The present documentation therefore makes the precise local claim: *no new axiom is introduced by* ***`Reflection.lean`***.

1.15 Architectural Significance

The development establishes a new path in CGJteamLab:

Hilbert geometry ⟶ geometrically defined reflection ⟶ point permutation ⟶ group word.

This is distinct from two earlier proof mechanisms:

- a Pappus-style relabelling argument, where the same points are read in a different order;
- direct Hilbert congruence proofs, where no transformation object is constructed.

Here the transformation is a first-class object derived from the Hilbert geometry. This is the feature required before Coxeter words and pairwise-generator relations can be studied formally.

1.16 Next Checkpoint

No further material should be added to `Reflection.lean` merely to anticipate later needs.

The next experimental module is

```
CGJteamLab/Coxeter/ReflectionWords.lean
```

with the initial task of composing two generators

$$r_l r_m.$$

Only after this composition layer is stable should the project attempt the second Coxeter relation

$$(r_l r_m)^{p_{lm}} = 1$$

for intersecting axes, and the infinite-period case associated with parallel axes.

1.17 Conclusion

The first Coxeter generator has been reconstructed rather than postulated.

Starting from Hilbert incidence, order, congruence, midpoint, and right-angle geometry, the development proves existence and uniqueness of line reflection, promotes the resulting relation to a permutation of the point set, and establishes

$$\boxed{r_l^2 = 1}.$$

This provides the first formal bridge from the synthetic Hilbert language of CGJteamLab to a generators-and-relations language of geometric transformations.

Chapter 2

Words in Reflection Generators

2.1 Purpose of the Module

The module

```
CGJteamLab/Coxeter/ReflectionWords.lean
```

is the first layer in which the geometric reflections constructed in `Reflection.lean` are treated as generators of words.

The previous chapter produced, for every reflection axis l, an equivalence

$$r_l : \text{Point} \simeq \text{Point}$$

satisfying

$$r_l^2 = 1.$$

The present module adds no new geometry. Its role is algebraic packaging: it composes reflection generators, iterates their products, and states the pairwise Coxeter relation in a form that later geometric modules can prove.

Thus the formal transition is

$$\text{geometric reflection} \longrightarrow \text{permutation} \longrightarrow \text{word in two generators.}$$

2.2 Product of Two Reflection Generators

For two axes `axis1` and `axis2`, the basic product is

```
noncomputable def reflectionProduct
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis1 axis2 : ReflectionAxis Geo) :
    Equiv Geo.Point Geo.Point :=
  (lineReflectEquiv Geo axis1).trans
    (lineReflectEquiv Geo axis2)
```

The order convention is important. On points,

$$\text{reflectionProduct}(r_1, r_2)(P) = r_2(r_1(P)).$$

This is recorded by the simplification theorem

```
@[simp]
theorem reflectionProduct_apply
    ...
    (P : Geo.Point) :
    reflectionProduct Geo axis1 axis2 P =
      lineReflect Geo axis2
        (lineReflect Geo axis1 P)
```

Hence `axis1` acts first and `axis2` second.

This convention is fixed once and used throughout the Coxeter layer. It is therefore possible to read expressions such as

$$r_1 r_2, \qquad (r_1 r_2)^2, \qquad (r_1 r_2)^p$$

directly in the Lean source without repeatedly reopening the underlying definition.

2.3 The Coincident-Axis Check

The first sanity check is the case in which the two axes coincide.

```
@[simp]
theorem reflectionProduct_self_apply
    ...
    (axis : ReflectionAxis Geo)
    (P : Geo.Point) :
    reflectionProduct Geo axis axis P = P
```

This theorem is an immediate consequence of `lineReflect_involutive`. It confirms that the algebraic product layer is synchronized with the previously proved geometric relation

$$r_l^2 = 1.$$

The result is small, but architecturally important: the word layer is not an independent abstract group model. It is built directly on the actual Hilbert reflection transformations.

2.4 Powers of a Reflection Product

The module next introduces iteration of the two-generator product:

```
noncomputable def reflectionProductPow
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis1 axis2 : ReflectionAxis Geo)
    (n : Nat) :
    Equiv Geo.Point Geo.Point := ...
```

The associated normalization theorems are

```
reflectionProductPow_zero
reflectionProductPow_succ
reflectionProductPow_one
```

They provide the formal recursion rules needed later to reduce a numerical Coxeter exponent to an explicit word.

Schematically,

$$(r_1 r_2)^0 = 1,$$

and

$$(r_1 r_2)^{n+1} = (r_1 r_2)^n (r_1 r_2).$$

At this stage no claim is made about the order of $r_1 r_2$. The module only constructs the word. Its period must come from geometry of the pair of axes.

2.5 The Pairwise Coxeter Relation

The central definition is

```
def ReflectionPairRelation
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis1 axis2 : ReflectionAxis Geo)
    (p : Nat) : Prop :=
  reflectionProductPow Geo axis1 axis2 p =
    Equiv.refl Geo.Point
```

This is the direct formal counterpart of the pairwise Coxeter relation

$$(r_i r_j)^{p_{ij}} = 1.$$

The definition deliberately contains no geometric hypothesis on the axes. It separates two logically different layers:

word construction ↓ `ReflectionPairRelation`	versus	geometry of the axes ↓ proof of a particular exponent p.

The next chapter supplies the first nontrivial instance: perpendicular axes imply $p = 2$.

2.6 Why the Word Layer is Separate

It would have been possible to define the special relation for perpendicular axes directly in the geometric module. The separate word layer is more useful.

First, it fixes the algebraic language before any particular configuration is studied. Second, the same definitions can be reused for intersecting axes with other finite periods and for parallel axes, where no finite period is expected. Third, the final theorem can be stated in the same form as a Coxeter presentation rather than as an ad hoc pointwise identity.

The architecture is therefore

$$\texttt{Reflection.lean}$$
$$\downarrow$$
$$r_l \in \mathrm{Equiv}(\mathrm{Point})$$
$$\downarrow$$
$$\texttt{ReflectionWords.lean}$$
$$\downarrow$$
$$r_1 r_2,\ (r_1 r_2)^n,\ \texttt{ReflectionPairRelation}$$
$$\downarrow$$
$$\texttt{CoxeterRelations.lean}.$$

2.7 Axiomatic Status

The module introduces no new geometric axiom and no new geometric construction. It only packages previously constructed reflections as equivalences and composes them.

The noncomputability is inherited from `lineReflectEquiv`, whose underlying point map ultimately uses `Classical.choose` after geometric existence and uniqueness have already been proved.

The exact transitive axiom set of later final theorems should be recorded separately by `#print axioms`. The local statement for this module is simply that no new foundational assumption is added by the word layer.

2.8 Conclusion

The module completes the passage from a single reflection generator to the language required for pairwise Coxeter relations:

$$\boxed{r_l^2 = 1 \quad \Longrightarrow \quad r_1 r_2 \quad \Longrightarrow \quad (r_1 r_2)^p \quad \Longrightarrow \quad \texttt{ReflectionPairRelation}.}$$

What remains is no longer to define the word, but to derive its period from the geometry of the reflecting axes.

Chapter 3

Relations Between Two Reflection Generators

3.1 Purpose of the Chapter

This chapter records the first genuine relation between two distinct geometric reflection generators.

The target is the Coxeter case

$$p_{12} = 2.$$

Geometrically, this is the case in which the two reflecting axes are perpendicular. The final theorem is

$$(r_1 r_2)^2 = 1.$$

The proof is not inserted as an abstract group identity. It is reconstructed from the Hilbert geometry of the two axes and from the previously proved metric behavior of line reflection.

Two Lean modules participate:

```
CGJteamLab/Coxeter/ReflectionIsometry.lean
CGJteamLab/Coxeter/CoxeterRelations.lean
```

The first establishes the metric behavior of a single reflection. The second packages perpendicular axes, proves the conjugation identity, and deduces the Coxeter relation.

3.2 Reflection as a Synthetic Isometry

The decisive theorem of `ReflectionIsometry.lean` is

```
theorem lineReflect_preserves_congruence
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis : ReflectionAxis Geo)
    (P Q : Geo.Point) :
    Geo.Congruent
      P Q
      (lineReflect Geo axis P)
```

```
      (lineReflect Geo axis Q)
```

Thus a line reflection preserves every segment congruence.

This is derived synthetically, without introducing coordinates or a numerical distance function. The proof separates several geometric configurations according to the perpendicular feet of the two points and uses the already established Hilbert congruence and right-angle theory.

Once segment congruence is available, the relation module extends the isometry package to the structural relations needed by conjugation:

```
lineReflect_preserves_between
lineReflect_preserves_collinear
lineReflect_preserves_noncollinear
lineReflect_preserves_angle
lineReflect_preserves_right_angle
```

The logical chain is

segment congruence $\longrightarrow$ betweenness and collinearity $\longrightarrow$ SSS $\longrightarrow$ angle congruence $\longrightarrow$ right-angle preservati

This is precisely the transformation-theoretic package required to move a reflection configuration through another reflection.

3.3 Perpendicular Reflection Axes

Perpendicularity of the two reflection axes is represented by explicit witness data:

```
structure ReflectionAxesPerpendicular
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis1 axis2 : ReflectionAxis Geo) where
  O : Geo.Point
  U : Geo.Point
  V : Geo.Point
  hO1 : HilbertIncidence.OnLine O axis1.carrier
  hO2 : HilbertIncidence.OnLine O axis2.carrier
  hU1 : HilbertIncidence.OnLine U axis1.carrier
  hV2 : HilbertIncidence.OnLine V axis2.carrier
  hOU : Not (O = U)
  hOV : Not (O = V)
  hNonCol : Not (PrimCollinear Geo U O V)
  hRight : HilbertRightAngle Geo U O V
```

The certificate makes all hidden diagrammatic information explicit. The axes meet at O; the points U and V determine nondegenerate directions on the two carriers; and $\angle UOV$ is right.

Perpendicularity is then shown to be symmetric by `reflectionAxesPerpendicular_symm`.

3.4 Setwise Preservation of the Perpendicular Axis

The first geometric consequence is that reflection in one axis preserves the other perpendicular axis setwise.

The key theorems are

```
perpendicular_axes_second_off_first

perpendicular_axes_second_point_perpendicular_first

lineReflect_preserves_perpendicular_axis_second

lineReflect_preserves_perpendicular_axis_first
```

Their geometric content is simple but formally important.

Let $P \in l_2$ with $P \neq O$. Then $P \notin l_1$, otherwise the two carriers would share two distinct points and therefore coincide, contradicting the nondegenerate right-angle configuration.

Moreover OP is perpendicular to l_1. Therefore O is the perpendicular foot used in the reflection of P across l_1. The midpoint condition gives

$$P - O - r_1(P),$$

so $r_1(P)$ remains on the carrier l_2.

Hence

$$r_1(l_2) = l_2, \qquad r_2(l_1) = l_1.$$

This setwise invariance is necessary, but by itself it does not yet prove that the two reflections commute.

3.5 Transport of Midpoints and Perpendiculars

The conjugation argument requires more than preservation of the axis. A complete reflection witness consists of a perpendicular foot together with a midpoint condition.

The theorem

```
theorem lineReflect_preserves_midpoint
    ...
    (hMid : HilbertIsMidpoint Geo M A B) :
    HilbertIsMidpoint
      Geo
      (lineReflect Geo axis M)
      (lineReflect Geo axis A)
      (lineReflect Geo axis B)
```

uses preservation of strict betweenness and of segment congruence.

The second transport theorem is

```
theorem perpendicular_axes_transport_second_perpendicular
    ...
    (hPerp :
      PerpendicularToAxis Geo axis2 H P) :
    PerpendicularToAxis
      Geo
      axis2
      (lineReflect Geo axis1 H)
      (lineReflect Geo axis1 P)
```

Here the invariant carrier l_2 remains the target axis, while the foot, the off-axis point, and the right-angle witness are all transported through r_1.

This is the technical core of the conjugation proof.

3.6 Conjugation

The central pointwise theorem is

```
theorem perpendicular_axes_conjugation_pointwise
    ...
    (P : Geo.Point) :
    lineReflect Geo axis1
        (lineReflect Geo axis2
          (lineReflect Geo axis1 P)) =
      lineReflect Geo axis2 P
```

In mathematical notation,

$$r_1 r_2 r_1 = r_2.$$

The proof splits according to whether P lies on l_2.

If $P \in l_2$, then $r_1(P) \in l_2$, so r_2 fixes both relevant points and the conclusion reduces to $r_1^2 = 1$.

Assume now $P \notin l_2$. Reflect $r_1(P)$ in l_2, obtaining a perpendicular foot H and midpoint configuration

$$r_1(P) - H - r_2(r_1(P)).$$

Apply r_1 to the whole configuration. Preservation of the perpendicular data and of the midpoint gives

$$P - r_1(H) - r_1 r_2 r_1(P),$$

with $r_1(H) \in l_2$ and the required right-angle condition.

Therefore $r_1 r_2 r_1(P)$ satisfies the defining relation `IsLineReflection` for the original point P and the axis l_2.

But the reflected point in a fixed axis is unique. Hence

$$r_1 r_2 r_1(P) = r_2(P).$$

This is the exact point where the earlier theorem `line_reflection_unique` re-enters the Coxeter development.

3.7 Commutation

Applying the conjugation identity to $r_1(P)$ and using $r_1^2 = 1$ gives

```
theorem perpendicular_axes_reflections_commute_pointwise
    ...
    (P : Geo.Point) :
    lineReflect Geo axis1
        (lineReflect Geo axis2 P) =
      lineReflect Geo axis2
        (lineReflect Geo axis1 P)
```

Thus

$$r_1 r_2 = r_2 r_1$$

pointwise.

The equality is also packaged at the permutation level:

```
theorem perpendicular_axes_reflectionProduct_commute
    ...
    reflectionProduct Geo axis1 axis2 =
      reflectionProduct Geo axis2 axis1
```

This is the permutation-level form of the classical fact that reflections in perpendicular lines commute.

3.8 The Square of the Product

From the conjugation identity one obtains directly

```
theorem perpendicular_axes_reflectionProduct_square_apply
    ...
    (P : Geo.Point) :
    reflectionProduct Geo axis1 axis2
        (reflectionProduct Geo axis1 axis2 P) =
      P
```

The pointwise result is then lifted to an equality of equivalences:

```
theorem perpendicular_axes_reflectionProductPow_two
    ...
    reflectionProductPow Geo axis1 axis2 2 =
      Equiv.refl Geo.Point
```

The proof of this wrapper is intentionally thin. All substantive geometry has already been completed before this stage.

3.9 The Coxeter Relation $p_{12} = 2$

The final theorem of the present checkpoint is

```
theorem perpendicular_axes_coxeter_relation_two
    [HilbertIncidence Geo]
    [HilbertCongruence Geo]
    (axis1 axis2 : ReflectionAxis Geo)
    (hAxes :
      ReflectionAxesPerpendicular Geo axis1 axis2) :
    ReflectionPairRelation
      Geo
      axis1 axis2
      2
```

Unfolding `ReflectionPairRelation`, this is exactly

$$\boxed{(r_1 r_2)^2 = 1}.$$

Thus the first nontrivial pairwise Coxeter exponent has been derived from the geometry of the mirrors:

$$\boxed{l_1 \perp l_2 \Longrightarrow p_{12} = 2.}$$

The theorem is stronger architecturally than an abstract presentation assumption. The generators themselves were reconstructed from Hilbert geometry, and the exponent 2 is derived from the geometric configuration of their axes.

3.10 Dependency Architecture

The completed proof may be summarized by the following DAG:

$$
\begin{array}{c}
\text{Hilbert incidence, order, congruence} \\
\Downarrow \\
\texttt{IsLineReflection} \longrightarrow \texttt{lineReflect} \longrightarrow r_l^2 = 1 \\
\Downarrow \\
\texttt{lineReflect\_preserves\_congruence} \\
\Downarrow \\
\text{betweenness, collinearity, angles, right angles} \\
\Downarrow \\
\texttt{ReflectionAxesPerpendicular} \\
\Downarrow \\
r_1(l_2) = l_2, \quad r_2(l_1) = l_1 \\
\Downarrow \\
\text{midpoint and perpendicular transport} \\
\Downarrow \\
r_1 r_2 r_1 = r_2 \\
\Downarrow \\
r_1 r_2 = r_2 r_1 \\
\Downarrow \\
(r_1 r_2)^2 = 1 \\
\Downarrow \\
\boxed{\texttt{ReflectionPairRelation 2}}.
\end{array}
$$

This is the first complete passage in the project from synthetic geometry to a nontrivial relation among several Coxeter generators.

3.11 Axiomatic Status

The two modules documented here introduce no new declared geometric axiom and no `sorry`.

The public relation theorem is stated with only `HilbertIncidence` and `HilbertCongruence` instances. The actual transitive axiom footprint of the final theorem should still be recorded explicitly with

```
#print axioms Geometry.perpendicular_axes_coxeter_relation_two
```

when the Coxeter layer is audited.

The present documentation therefore makes the local claim that the $p = 2$ relation has been proved inside the already established Hilbert infrastructure without adding a new Coxeter-specific geometric axiom.

3.12 Conclusion

The completed chapter establishes the chain

$$\boxed{\text{perpendicular mirrors} \Longrightarrow r_1 r_2 r_1 = r_2 \Longrightarrow r_1 r_2 = r_2 r_1 \Longrightarrow (r_1 r_2)^2 = 1.}$$

In the language prepared by `ReflectionWords.lean`, this becomes

`ReflectionPairRelation` 2.

The next mathematical problem is the first nonperpendicular finite case. Rather than introducing numerical angle measure immediately, the next chapter uses an equilateral triangle and its three median reflection axes to derive synthetically

$$r_a r_b r_a = r_b r_a r_b$$

and hence

$$(r_a r_b)^3 = 1.$$

Only after this $p = 3$ configuration has been isolated should the document open the general finite-period problem for arbitrary intersecting mirrors.

Chapter 4

The Coxeter Relation $p = 3$: Equilateral Median Reflections

4.1 Purpose of the Chapter

The perpendicular case $p = 2$ is the first nontrivial pairwise Coxeter relation, but it is still exceptional: the two reflections commute. The present chapter records the first genuinely noncommutative case obtained in the synthetic Hilbert development.

The target is

$$p_{ab} = 3.$$

Rather than introducing a numerical angle of $60°$, the construction uses an equilateral triangle and its three median reflection axes. If a, b, c denote these axes, the synthetic geometry proves the two carrier transports

$$r_b(a) = c, \qquad r_a(b) = c.$$

The general conjugation theorem then yields

$$r_b r_a r_b = r_c, \qquad r_a r_b r_a = r_c,$$

and therefore the braid relation

$$\boxed{r_a r_b r_a = r_b r_a r_b}.$$

Using only involutivity of the two reflections,

$$r_a^2 = r_b^2 = 1,$$

this gives

$$\boxed{(r_a r_b)^3 = 1}.$$

This chapter is therefore the first completed $p = 3$ geometric relation in the Coxeter layer.

The relevant source remains

```
CGJteamLab/Coxeter/CoxeterRelations.lean
```

The experimental development was first stabilized in the cumulative general-test chain and then transferred to the main relation module.

4.2 Why the $p = 3$ Step is Structurally Different

For perpendicular axes the decisive geometric fact was setwise preservation: reflection in one axis preserves the other axis. This produced

$$r_1 r_2 r_1 = r_2,$$

hence commutation and the square relation.

The $p = 3$ case cannot be obtained by repeating that argument. Reflection in one median axis of an equilateral triangle does not preserve a second median axis. It sends that axis to the third one.

Thus the correct geometric pattern is no longer

$$r_a(b) = b,$$

but rather

$$r_a(b) = c.$$

This is the first place where the Coxeter layer needs a genuine transport theory for reflection axes. The new proof architecture is

$$\begin{array}{c} \text{reflection of points} \\ \Downarrow \\ \text{reflection of a carrier line} \\ \Downarrow \\ \text{conjugation of line reflections} \\ \Downarrow \\ \text{braid relation.} \end{array}$$

The passage from $p = 2$ to $p = 3$ is therefore not merely a change of exponent. It introduces the mechanism by which one reflection transports another reflection generator to a different generator.

4.3 No Numerical Angle is Introduced

The construction is deliberately synthetic.

No distance function, coordinate system, scalar product, trigonometric function, or numerical angle measure is introduced. In particular, the proof does not define the angle between a and b to be 60°, and it does not use the analytic identity

$$2\cos(\pi/3) = 1.$$

Instead, the order-three relation is extracted from a finite geometric configuration whose symmetries can be reconstructed using the already available Hilbert notions:

- segment congruence,
- midpoint,
- collinearity and betweenness,

- right angle and perpendicularity,
- line reflection,
- preservation of midpoint data by reflection,
- uniqueness of a midpoint.

The equilateral triangle is therefore not being used as an analytic model of an angle. It is used as a synthetic incidence–congruence configuration that forces the required permutation of median axes.

4.4 The Equilateral Configuration

Let A, B, C be noncollinear points satisfying the three apex forms of the equilateral side condition:

$$AB \cong AC, \qquad BA \cong BC, \qquad CB \cong CA.$$

The Lean hypotheses are oriented in exactly this way because the median-axis construction is invoked once from each vertex.

Let

$$M_A = \mathrm{mid}(B, C), \qquad M_B = \mathrm{mid}(A, C), \qquad M_C = \mathrm{mid}(B, A).$$

The three reflection axes are then constructed as the median carriers

$$a = AM_A, \qquad b = BM_B, \qquad c = CM_C.$$

The choice

$$M_C = \mathrm{mid}(B, A)$$

rather than initially orienting the midpoint as $\mathrm{mid}(A, B)$ is only a formal endpoint convention. The public midpoint relation is symmetric, and the existing theorem

```
MidpointSymmetry
```

is used when the opposite orientation is required.

4.5 The Median Axis of an Isosceles Triangle

The geometric constructor underlying all three axes is

```
isosceles_midpoint_axis_swaps_base
```

Its input consists of an isosceles triangle with apex A, base BC, and a midpoint M of BC. The theorem constructs a reflection axis through A and M and proves that the associated line reflection fixes the apex and midpoint while swapping the base endpoints:

$$r(A) = A, \qquad r(M) = M, \qquad r(B) = C, \qquad r(C) = B.$$

Schematically, the theorem has the form

```
theorem isosceles_midpoint_axis_swaps_base
    ...
    (hABC : Not (Collinear Geo A B C))
    (hABAC : Geo.Congruent A B A C)
    (hMid : HilbertIsMidpoint Geo M B C) :
    Exists fun axis : ReflectionAxis Geo =>
      ...
```

The proof itself depends on the synthetic right-angle theorem for the median of an isosceles triangle:

```
isosceles_midpoint_right_angle
```

Thus the median is promoted to an actual reflection axis without assuming a global symmetry of the triangle as a primitive transformation.

Applying the theorem at the three vertices gives:

axis	fixed points	swapped vertices
a	A, M_A	$B \leftrightarrow C$
b	B, M_B	$A \leftrightarrow C$
c	C, M_C	$B \leftrightarrow A.$

This finite table is the geometric engine of the $p = 3$ proof.

4.6 Transporting an Entire Carrier Line

Pointwise reflection of the vertices is not yet enough for conjugation. Conjugation requires knowledge of the image of the whole carrier of a reflection axis.

The relation

```
def ReflectionMapsLine
    ...
    (mirror : ReflectionAxis Geo)
    (source target : Geo.Line) : Prop :=
  forall P : Geo.Point,
    HilbertIncidence.OnLine P source <->
      HilbertIncidence.OnLine
        (lineReflect Geo mirror P)
        target
```

encodes exact setwise transport of one line onto another.

The meaning of

$$\mathrm{ReflectionMapsLine}(b, a, c)$$

is

$$P \in a \quad \Longleftrightarrow \quad r_b(P) \in c.$$

This is stronger than merely proving that two selected points of a land on c. The selected points are instead used as a criterion from which the full line transport is derived.

4.7 The Two-Point Criterion

The helper

```
reflectionMapsLine_of_two_points
```

is the bridge from point geometry to carrier geometry.

Suppose X, Y are distinct points on the source carrier and their reflected images both lie on the target carrier. Incidence uniqueness then forces the reflected source line to coincide with the target line, giving

$$\text{ReflectionMapsLine}\,.$$

In the $p = 3$ proof the source points are chosen to be the two defining points of a median carrier:

$$A, M_A \in a$$

or

$$B, M_B \in b.$$

Thus the main problem becomes the precise determination of the images of the midpoints M_A and M_B.

4.8 Midpoint Transport Under Reflection

The already established reflection-isometry layer provides

```
lineReflect_preserves_midpoint
```

which transports a midpoint configuration through any line reflection.

From

$$M_A = \text{mid}(B, C)$$

and reflection in b, we obtain

$$r_b(M_A) = \text{mid}\big(r_b(B), r_b(C)\big).$$

But the median reflection b fixes B and swaps C with A:

$$r_b(B) = B, \qquad r_b(C) = A.$$

Hence

$$r_b(M_A)$$

is a midpoint of BA.

The designated point M_C is also a midpoint of BA. Therefore midpoint uniqueness gives

$$\boxed{r_b(M_A) = M_C}.$$

The local uniqueness lemma used here is

```
hilbert_midpoint_unique_local
```

with statement of the form

```
theorem hilbert_midpoint_unique_local
    ...
    (hM : HilbertIsMidpoint Geo M A B)
    (hN : HilbertIsMidpoint Geo N A B) :
    M = N
```

This is one of the key synthetic moves in the chapter. No coordinates are needed to calculate the reflected midpoint. Its image is identified by the universal geometric property of being the midpoint of the reflected endpoints.

4.9 First Carrier Transport: $r_b(a) = c$

The first genuine $p = 3$ theorem is

```
equilateral_median_axis_b_maps_a_to_c
```

The proof uses the two defining points of a:

$$A,\ M_A.$$

Reflection in b gives

$$r_b(A) = C$$

and, by midpoint transport and uniqueness,

$$r_b(M_A) = M_C.$$

Both C and M_C lie on the carrier c. Since $A \neq M_A$, the two-point criterion yields

$$\boxed{r_b(a) = c}.$$

In Lean the conclusion is represented as

```
ReflectionMapsLine Geo b a.carrier c.carrier
```

rather than as an equality between primitive line-image objects.

This theorem is the first place in the project where reflection in one Coxeter generator transports the carrier of a second generator to the carrier of a third.

4.10 General Conjugation by Carrier Transport

The carrier transport theorem feeds directly into the general conjugation principle

```
lineReflect_conjugation_pointwise
```

whose geometric content is: if reflection in m sends the carrier of s onto the carrier of t, then

$$r_m r_s r_m = r_t.$$

More precisely, from

$$r_m(s) = t$$

the theorem proves pointwise

$$r_m(r_s(r_m(P))) = r_t(P).$$

Applying this with

$$m = b, \qquad s = a, \qquad t = c$$

gives

$$\boxed{r_b r_a r_b = r_c}.$$

The corresponding Lean theorem is

```
equilateral_median_reflection_conjugation_bab_eq_c
```

This isolates the group-theoretic consequence of the first carrier transport before any braid identity is used.

4.11 Second Carrier Transport on the Same Axes

To derive the braid relation, it is essential that both conjugation identities refer to the *same* existentially chosen axes a, b, c. Constructing a second unrelated triple of axes would leave an additional identification problem.

The synchronization theorem

```
equilateral_median_axes_two_transports
```

therefore constructs one triple a, b, c and proves simultaneously

$$r_b(a) = c$$

and

$$r_a(b) = c.$$

The second transport is proved in exact analogy with the first.

Since a fixes A and swaps B with C,

$$r_a(A) = A, \qquad r_a(C) = B.$$

Transporting

$$M_B = \mathrm{mid}(A, C)$$

through r_a shows that $r_a(M_B)$ is a midpoint of AB.

The designated midpoint M_C was initially oriented as a midpoint of BA. The theorem

`MidpointSymmetry`

converts this to the required midpoint of AB, after which uniqueness gives

$$r_a(M_B) = M_C.$$

Together with

$$r_a(B) = C$$

the two-point criterion yields

$$\boxed{r_a(b) = c}.$$

4.12 The Braid Relation

The two carrier transports now give two conjugation identities on the same axes:

$$r_b r_a r_b = r_c,$$

$$r_a r_b r_a = r_c.$$

Comparing them immediately gives

$$\boxed{r_a r_b r_a = r_b r_a r_b.}$$

The Lean theorem is

```
equilateral_median_reflections_braid
```

and its final proof step is simply the transitivity of equality through the common reflection r_c.

This is the first genuine braid relation in the project.

It is also the first relation in the Coxeter layer in which the two generators do not collapse to a commutative pair. For $p = 2$, the geometry forced

$$r_1 r_2 = r_2 r_1.$$

For $p = 3$, the correct reduced relation is instead

$$r_a r_b r_a = r_b r_a r_b.$$

This is the standard braid form associated with the rank-two Coxeter system of type A_2, equivalently $I_2(3)$.

4.13 From the Braid Relation to Order Three

Once the braid relation is available, no further geometry is needed.

The final theorem of the present chain is

```
equilateral_median_reflections_product_order_three
```

and proves pointwise

$$(r_a r_b)^3(P) = P.$$

Expanded literally, the left-hand side is

$$r_a r_b r_a r_b r_a r_b(P).$$

Using

$$r_a r_b r_a = r_b r_a r_b$$

on the first three factors gives

$$r_b r_a r_b r_b r_a r_b(P).$$

Now involutivity cancels the adjacent equal reflections:

$$r_b r_a \underbrace{r_b r_b}_{1} r_a r_b(P) = r_b \underbrace{r_a r_a}_{1} r_b(P) = \underbrace{r_b r_b}_{1}(P) = P.$$

Thus

$$\boxed{(r_a r_b)^3 = 1}.$$

The proof is intentionally separated from the geometric part. All geometric work has already been completed by the carrier transports and conjugation theorems. The order-three result is then a short word calculation using only the braid relation and involutivity.

4.14 Relation Between the Two Forms of the $p = 3$ Relation

For involutions $a^2 = b^2 = 1$, the two familiar forms

$$aba = bab$$

and

$$(ab)^3 = 1$$

encode the same rank-two Coxeter relation.

The present development reaches them in the geometrically natural order:

$$\text{carrier transport} \Longrightarrow \text{conjugation} \Longrightarrow aba = bab \Longrightarrow (ab)^3 = 1.$$

This order is important for the formal architecture. The braid identity is not guessed algebraically and then checked. It is forced by the fact that both a and b, under conjugation by the other, are transported to the same third geometric reflection c.

The third median axis is therefore not auxiliary decoration. It is the geometric witness that explains why the braid relation holds.

4.15 Dependency Architecture

The $p = 3$ chain may be summarized as follows:

$$
\begin{array}{c}
\texttt{lineReflect\_preserves\_midpoint} \\
\Downarrow \\
\texttt{isosceles\_midpoint\_axis\_swaps\_base} \\
\Downarrow \\
a, b, c \text{ median reflection axes} \\
\Downarrow \\
\texttt{hilbert\_midpoint\_unique\_local} \\
\Downarrow \\
r_b(M_A) = M_C, \quad r_a(M_B) = M_C \\
\Downarrow \\
\texttt{reflectionMapsLine\_of\_two\_points} \\
\Downarrow \\
r_b(a) = c, \quad r_a(b) = c \\
\Downarrow \\
\texttt{lineReflect\_conjugation\_pointwise} \\
\Downarrow \\
r_b r_a r_b = r_c, \quad r_a r_b r_a = r_c \\
\Downarrow \\
\boxed{r_a r_b r_a = r_b r_a r_b} \\
\Downarrow \\
r_a^2 = r_b^2 = 1 \\
\Downarrow \\
\boxed{(r_a r_b)^3 = 1}.
\end{array}
$$

The crucial new middle layer, absent from the original $p = 2$ proof, is

$$\text{point transport} \longrightarrow \text{carrier transport} \longrightarrow \text{reflection conjugation}.$$

This layer is expected to remain useful beyond the equilateral case.

4.16 Axiomatic Status

No new Coxeter-specific geometric axiom is introduced in the $p = 3$ construction.

The argument is built over the already established Hilbert incidence and congruence infrastructure together with the derived reflection machinery. The local theorem statements use the same typeclass assumptions as the preceding reflection relation layer.

As elsewhere in the project, the exact transitive axiom footprint of the final public theorem should be recorded separately by Lean audit, for example with

```
#print axioms
  Geometry.equilateral_median_reflections_product_order_three
```

after the production module has been rebuilt at the documentation checkpoint.

Accordingly, the precise local claim of this chapter is that no new geometric axiom or `sorry` was introduced in order to obtain the $p = 3$ relation.

4.17 Current Formal Status and Word-Layer Packaging

The geometric relation currently ends with the pointwise theorem

$$(r_a r_b)^3(P) = P$$

for every point P.

This is already the substantive synthetic $p = 3$ result.

There remains, however, one small algebraic packaging step before the chapter can claim the exact word-layer proposition

`ReflectionPairRelation` 3.

The distinction is deliberate. The present chapter does not identify the pointwise theorem with the packaged relation until that final bridge has been checked in Lean against the definitions of `reflectionProduct`, `reflectionProductPow`, and `ReflectionPairRelation` from `ReflectionWords.lean`.

Thus the current status is

$$\boxed{\text{synthetic geometry} \Longrightarrow aba = bab \Longrightarrow (ab)^3 = 1 \text{ pointwise}}$$

with the final word-layer export as the next local checkpoint.

4.18 Why This Chapter Matters for the General Program

The $p = 2$ case could still be viewed as a special orthogonality theorem. The $p = 3$ case demonstrates a broader mechanism.

The geometry need not preserve each generator individually. It may permute a finite family of reflection axes, and this permutation can be converted into conjugation identities among the corresponding reflection generators.

For the equilateral configuration,

$$a,\ b,\ c$$

form exactly such a finite family.

This suggests the next research question in a synthetic form:

> Which finite geometric configurations of reflection carriers force a prescribed finite period for a product of two reflection generators?

That question is deliberately more geometric than the analytic statement "the angle is π/p". The present chapter shows that at least for $p = 3$, the Coxeter exponent can be recovered from a purely synthetic configuration without first constructing numerical angle measure.

4.19 Conclusion

The completed $p = 3$ chain is

$$\boxed{\begin{array}{c}\text{equilateral triangle}\\ \Downarrow\\ \text{three median reflection axes } a, b, c\\ \Downarrow\\ r_b(a) = c, \quad r_a(b) = c\\ \Downarrow\\ r_b r_a r_b = r_c, \quad r_a r_b r_a = r_c\\ \Downarrow\\ r_a r_b r_a = r_b r_a r_b\\ \Downarrow\\ (r_a r_b)^3 = 1.\end{array}}$$

The proof is synthetic throughout. The exponent 3 is not imported from coordinates, trigonometry, or a numerical 60° angle. It is forced by the reflection geometry of the equilateral configuration.

The next immediate formal task is the small word-layer bridge to

`ReflectionPairRelation` 3.

After that checkpoint, the next mathematical chapter can address the general finite-period problem for intersecting reflection axes.

Appendix A

Source Map: Coxeter–Moser

The supplement currently uses the following parts of Coxeter–Moser as conceptual control.

A.1 How the Source Book Is Being Used

The source book is first of all a book about presentations of discrete groups. In the preface to the first edition, Coxeter and Moser describe their original ambition as a catalogue of abstract definitions for finitely generated groups, later restricted to selected families. Groups generated by reflections in real Euclidean space occur as one of those families, not as the sole organizing subject of the book.

This distinction matters for the present project. We are not formalizing Coxeter–Moser chapter by chapter. We use their generators-and-relations language as a target vocabulary for a synthetic reconstruction:

$$\begin{gathered}\text{Coxeter–Moser direction}\\ \text{presentation} \longrightarrow \text{group} \longrightarrow \text{geometric realization in selected cases},\end{gathered}$$

whereas the present reflection program proceeds as

$$\begin{gathered}\text{CGJteamLab direction}\\ \text{Hilbert geometry} \longrightarrow \text{reflection transformations} \longrightarrow \text{words} \longrightarrow \text{relations}.\end{gathered}$$

The most important source-book checkpoint for this reversal is Chapter 9. There Coxeter–Moser treat finite groups with involutory generators and pairwise relations of the form

$$R_i^2 = 1, \qquad (R_i R_j)^{p_{ij}} = 1,$$

and realize them by reflections in hyperplanes of Euclidean n-space. For the present project this is a destination and consistency check, not the current proof method.

A.2 Section 1.1: Generators and Relations

This section supplies the abstract language of generators, words, relations, and presentations.

It also contains a distinction that is important for the Lean interface. If a generator satisfies

$$S^q = 1,$$

this alone shows only that its period divides q. Coxeter–Moser reserve the stronger presentation-level reading for the case in which the period is exactly q, distinguishing a relation that is merely satisfied from one that is fulfilled.

Consequently, a predicate asserting only

$$(r_i r_j)^p = 1$$

should not automatically be read as saying that the product has exact period p. The current `ReflectionPairRelation` layer should be audited with this distinction in mind before it is treated as a literal formal counterpart of a Coxeter–Moser period label.

A.3 Section 4.1: Cyclic and Dihedral Groups

This section provides the elementary geometric interpretation of dihedral groups in terms of rotations and reflections.

A.4 Section 4.3: Groups Generated by Reflections

This section connects words in reflection generators with paths through fundamental regions and explains how local geometric circuits produce defining relations.

A.5 Sections 9.1–9.3: Groups Generated by Reflections

These sections provide the general Coxeter-type presentation

$$r_i^2 = 1, \qquad (r_i r_j)^{p_{ij}} = 1,$$

the graphical notation, and the realization of generators by real affine reflections.

Appendix B

Lean Module Map

The current module architecture is

```
CGJteamLab/Coxeter/
  Reflection.lean
  ReflectionWords.lean
  ReflectionIsometry.lean
  CoxeterRelations.lean
  EuclidBenchmarks.lean        -- planned
```

The completed Coxeter layer currently consists of `Reflection.lean`, `ReflectionWords.lean`, `ReflectionIsometry.lean`, and `CoxeterRelations.lean`.

Checkpoint

Current completed milestone:

$$
\boxed{
\begin{array}{c}
\text{Hilbert line reflection} \\
\Downarrow \\
r_l^2 = 1 \\
\Downarrow \\
\text{words in reflection generators} \\
\Downarrow \\
\text{synthetic reflection isometry} \\
\Downarrow \\
l_1 \perp l_2 \\
\Downarrow \\
(r_1 r_2)^2 = 1 \\
\Downarrow \\
\texttt{ReflectionPairRelation}\ 2 \\
\Downarrow \\
\text{equilateral median reflection axes } a, b, c \\
\Downarrow \\
r_b(a) = c, \quad r_a(b) = c \\
\Downarrow \\
r_a r_b r_a = r_b r_a r_b \\
\Downarrow \\
(r_a r_b)^3 = 1 \quad \text{pointwise}
\end{array}
}
$$

The immediate formal checkpoint is to export the last pointwise theorem to

$$\texttt{ReflectionPairRelation}\ 3.$$

After that packaging step, the next mathematical target is the general finite-period case for intersecting axes, formulated synthetically before any numerical angle machinery is introduced.